Отчёт по лабораторной работе №3

Управляющие структуры

Дагделен Зейнап Реджеповна

Отчёт о выполнении лабораторной работы по теме «Управляющие структуры» на языке Julia. Освоено применение циклов, функций и сторонних пакетов для решения задач линейной алгебры и работы с матрицами.

Содержание

1 Введение **6**
1.1 Актуальность . . . 6
1.2 Цель работы . . . 6
1.3 Задачи . . . 6
1.4 Объект и предмет исследования . . . 7
1.5 Методы исследования . . . 7

2 Теоретическая часть **8**

3 Материалы и методы **9**

4 Выполнение лабораторной работы **10**

5 Результаты **13**
5.1 Примеры из теории . . . 13
5.2 Задание 1. Циклы, словарь, массив . . . 19
5.3 Задание 2. Условный оператор: чётное/нечётное . . . 23
5.4 Задание 3. Функция add_one . . . 24
5.5 Задание 4. map / broadcast: увеличение элементов матрицы . . . 24
5.6 Задание 5. Матрица A, A^3, замена столбца . . . 25
5.7 Задание 6. Матрица B и C = B⊠·B . . . 27
5.8 Задание 7. Матрицы $Z_1..Z_4$ . . . 28
5.9 Задание 8. Функция outer() и матрицы $A_1..A_5$ . . . 31
5.10 Задание 9. Решение СЛАУ Ax = y . . . 34
5.11 Задание 10. Матрица 6×10 . . . 37
5.12 Задание 11. Двойные суммы . . . 39
5.13 Выводы . . . 40

6 Обсуждение результатов **42**

7 Заключение **43**

Список литературы **44**

Список иллюстраций

4.1 Скрипт `lab3.jl` в редакторе кода 10
4.2 Запуск скрипта в терминале 11
4.3 Генерация `.qmd` и `.ipynb` через `tangle.jl` 12

Список таблиц

1 Введение

1.1 Актуальность

Управляющие конструкции — базовый инструмент программирования, необходимый для обхода матриц, организации вычислений и обработки результатов в задачах статистического анализа данных. Язык Julia предоставляет широкий набор управляющих структур и функций высшего порядка [4].

1.2 Цель работы

Освоить применение циклов, функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.

1.3 Задачи

1. Изучить циклы `while` и `for`, условный и тернарный операторы.
2. Освоить функции разных форм и функции высшего порядка.
3. Реализовать аналог функции `outer()` языка R.
4. Применить пакеты `DrWatson` и `Literate`.
5. Выполнить задания 1–11 методических указаний.

1.4 Объект и предмет исследования

Объект: язык Julia и его управляющие конструкции. **Предмет:** применение циклов, функций и сторонних пакетов для задач линейной алгебры и работы с матрицами.

1.5 Методы исследования

Анализ документации, программная реализация заданий, вычислительный эксперимент, генерация отчёта средствами `Literate`.

2 Теоретическая часть

Циклы. `while <условие> ... end` и `for <переменная> in <диапазон> ... end` используются для перебора индексируемых элементов [4].

Условный и тернарный операторы. `if a; b; else; c; end` эквивалентно `a ? b : c` [5].

Функции. Объявляются через `function ... end`, в одну строку (`f(x) = x^2`) или как анонимные (`x -> x^2`). По соглашению функции с `!` изменяют аргумент (`sort!`), без `!` — не изменяют (`sort`) [3].

Функции высшего порядка. `map(f, xs)` применяет `f` поэлементно; `broadcast` (кратко `f.(xs)`) — обобщение с трансляцией размерностей [5].

`outer(x, y, operation)` — аналог функции R: матрица C, где $C_{ij} = \text{operation}(x_i, y_j)$.

Пакеты. `DrWatson` [2] — организация проекта; `Literate` [1] — генерация отчётов (`.qmd`, `.ipynb`) из скрипта с разметкой; `Random` — генерация случайных чисел с воспроизводимостью.

Операционная система. Работа выполнялась в среде Ubuntu [9]. Для работы с командной строкой использовались базовые приёмы Bash [6—8].

3 Материалы и методы

ПО: Ubuntu, Julia 1.12 [4], пакеты `DrWatson` v2.19.1 [2], `Literate` v2.21.0 [1], `Random`.

4 Выполнение лабораторной работы

Работа выполнена в три этапа.

1. На рис. 4.1 представлен исходный файл `lab3.jl` в редакторе кода.

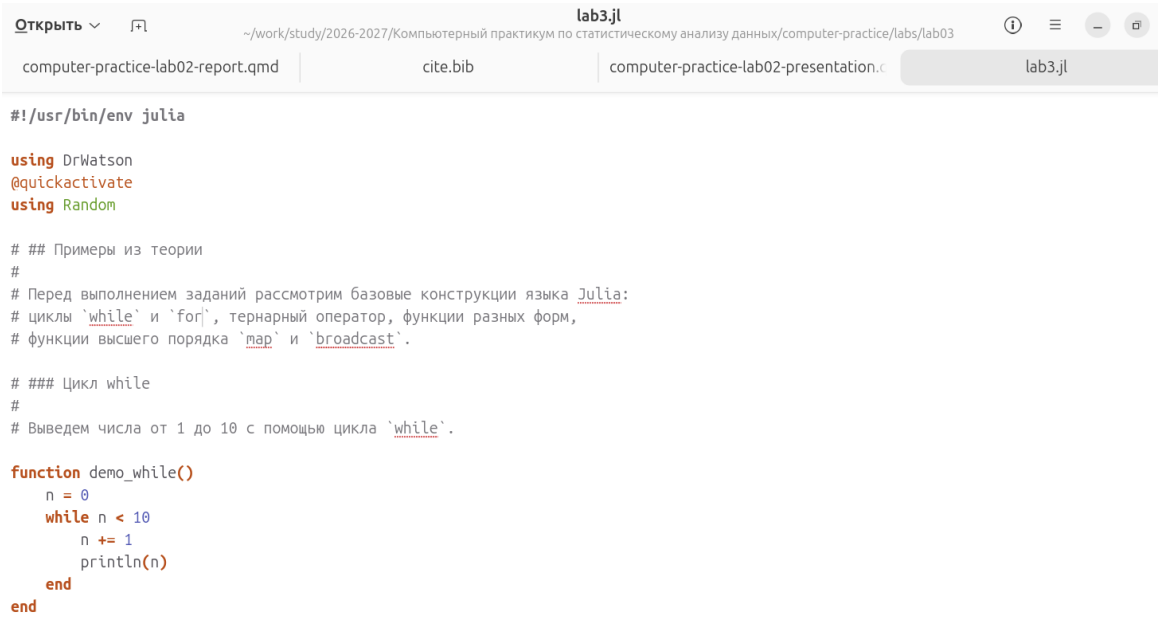


Рисунок 4.1: Скрипт `lab3.jl` в редакторе кода

2. На рис. 4.2 показан запуск скрипта командой `julia --project=. lab3.jl | tee output.txt`. Вывод направляется в терминал и одновременно сохраняется в файл `output.txt`. Видно, что скрипт выполняется без ошибок, выводятся результаты первых заданий.

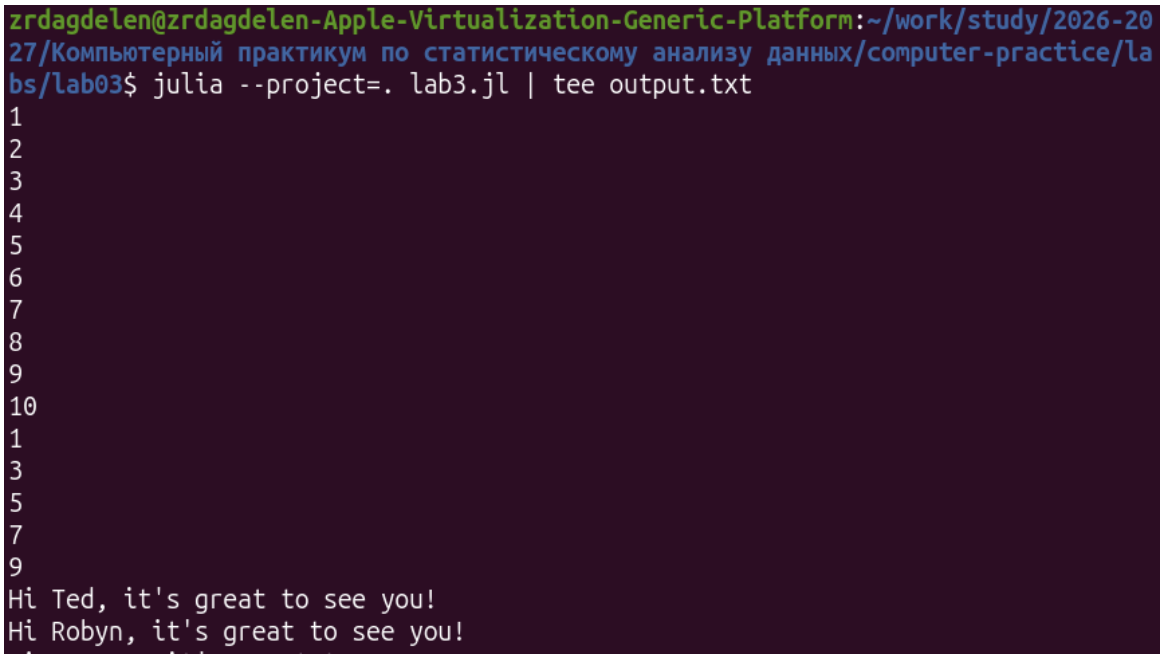


Рисунок 4.2: Запуск скрипта в терминале

3. На рис. 4.3 показан запуск генератора `tangle.jl`, который создаёт три файла: чистый скрипт `scripts/lab3/lab3.jl`, Quarto-документ `markdown/lab3/lab3.qmd` и Jupyter Notebook `notebooks/lab3/lab3.ipynb`.



Рисунок 4.3: Генерация `.qmd` и `.ipynb` через `tangle.jl`

Подробное описание всех заданий с листингами и результатами — в разделе 5.

5 Результаты

```
#!/usr/bin/env julia


using DrWatson
@quickactivate
using Random
```

5.1 Примеры из теории

Перед выполнением заданий рассмотрим базовые конструкции языка Julia: циклы `while` и `for`, тернарный оператор, функции разных форм, функции высшего порядка `map` и `broadcast`.

5.1.1 Цикл while

Выведем числа от 1 до 10 с помощью цикла `while`.

```
function demo_while()
    n = 0
    while n < 10
        n += 1
        println(n)
    end
```

```
end


demo_while()
```

```
1
2
3
4
5
6
7
8
9
10
```

5.1.2 Цикл for по диапазону

Тот же результат можно получить циклом `for` с шагом 2.

```
function demo_for_range()
    for k in 1:2:10
        println(k)
    end
end


demo_for_range()
```

```
1
3
5
```

```
7
9
```

5.1.3 Цикл for по массиву строк

Пройдёмся по массиву имён и выведем приветствие для каждого.

```
function demo_for_array()
    myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]
    for friend in myfriends
        println("Hi $friend, it's great to see you!")
    end
end


demo_for_array()
```

```
Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!
```

5.1.4 Тернарный оператор

Тернарный оператор позволяет записать условное выражение в одну строку.

```
function demo_ternary()
    x = 5
    y = 10
    println("(x > y) ? x : y = ", (x > y) ? x : y)
end
```

```
demo_ternary()
```

```
(x > y) ? x : y = 10
```

5.1.5 Функции

В Julia функцию можно объявить несколькими способами: через `function ... end`, в одну строку и как анонимную функцию.

```
function demo_functions()
    sayhi(name) = println("Hi $name, it's great to see you!")
    f2(x) = x^2

    sayhi3 = name -> println("Hi $name, it's great to see you!")
    f3 = x -> x^2

    sayhi("C-3PO")
    println("f2(42) = ", f2(42))
    sayhi3("BB-8")
    println("f3(42) = ", f3(42))
end

demo_functions()
```

```
Hi C-3PO, it's great to see you!
f2(42) = 1764
Hi BB-8, it's great to see you!
f3(42) = 1764
```

5.1.6 Функции с восклицательным знаком

По соглашению функции с `!` изменяют свой аргумент, без `!` — не изменяют. Сравним `sort` и `sort!`.

```
function demo_sort()
    v = [3, 5, 2]
    println("Исходный массив: ", v)
    println("sort(v) = ", sort(v))
    println("v после sort(v): ", v)
    sort!(v)
    println("v после sort!(v): ", v)
end

demo_sort()
```

```
Исходный массив: [3, 5, 2]
sort(v) = [2, 3, 5]
v после sort(v): [3, 5, 2]
v после sort!(v): [2, 3, 5]
```

5.1.7 Функции высшего порядка: map и broadcast

`map` применяет функцию к каждому элементу коллекции. `broadcast` делает то же самое, но с поддержкой трансляции размерностей.

```
function demo_map_broadcast()
    f_sq(x) = x^2
    println("map(f_sq, [1,2,3]) = ", map(f_sq, [1, 2, 3]))
    println("map(x -> x^3, [1,2,3]) = ", map(x -> x^3, [1, 2, 3]))
    println("broadcast(f_sq, [1,2,3]) = ", broadcast(f_sq, [1, 2, 3]))
```

```
    println("f_sq.([1,2,3]) = ", f_sq.([1, 2, 3]))
end


demo_map_broadcast()
```

```
map(f_sq, [1,2,3]) = [1, 4, 9]
map(x -> x^3, [1,2,3]) = [1, 8, 27]
broadcast(f_sq, [1,2,3]) = [1, 4, 9]
f_sq.([1,2,3]) = [1, 4, 9]
```

5.1.8 Матрица через генератор

Создадим матрицу через генератор и применим к ней функцию `f`.

```
function demo_matrix_gen()
    f(x) = x^2
    A = [i + 3*j for j in 0:2, i in 1:3]
    display(A)
    println("\nf(A) = A*A:")
    display(f(A))
end


demo_matrix_gen()
```

```
f(A) = A*A:

3×3 Matrix{Int64}:
 1  2  3
 4  5  6
 7  8  9
```

```
3×3 Matrix{Int64}:
  30   36   42
  66   81   96
 102  126  150
```

5.2 Задание 1. Циклы, словарь, массив

Требуется: 1. Вывести целые числа от 1 до 100 и их квадраты. 2. Создать словарь `squares` с парами «число → квадрат». 3. Создать массив `squares_arr` с квадратами чисел от 1 до 100.

```
function task1()
    squares = Dict{Int, Int}()
    i = 1
    while i <= 100
        squares[i] = i^2
        println("$i^2 = $(i^2)")
        i += 1
    end

    squares_arr = [k^2 for k in 1:100]
    println("\nsquares_arr (первые 10): ", squares_arr[1:10], " ...")
end

task1()
```

```
1^2 = 1
2^2 = 4
3^2 = 9
4^2 = 16
```

```
5^2 = 25
6^2 = 36
7^2 = 49
8^2 = 64
9^2 = 81
10^2 = 100
11^2 = 121
12^2 = 144
13^2 = 169
14^2 = 196
15^2 = 225
16^2 = 256
17^2 = 289
18^2 = 324
19^2 = 361
20^2 = 400
21^2 = 441
22^2 = 484
23^2 = 529
24^2 = 576
25^2 = 625
26^2 = 676
27^2 = 729
28^2 = 784
29^2 = 841
30^2 = 900
31^2 = 961
32^2 = 1024
33^2 = 1089
```

```
34^2 = 1156
35^2 = 1225
36^2 = 1296
37^2 = 1369
38^2 = 1444
39^2 = 1521
40^2 = 1600
41^2 = 1681
42^2 = 1764
43^2 = 1849
44^2 = 1936
45^2 = 2025
46^2 = 2116
47^2 = 2209
48^2 = 2304
49^2 = 2401
50^2 = 2500
51^2 = 2601
52^2 = 2704
53^2 = 2809
54^2 = 2916
55^2 = 3025
56^2 = 3136
57^2 = 3249
58^2 = 3364
59^2 = 3481
60^2 = 3600
61^2 = 3721
62^2 = 3844
```

63^2 = 3969

64^2 = 4096

65^2 = 4225

66^2 = 4356

67^2 = 4489

68^2 = 4624

69^2 = 4761

70^2 = 4900

71^2 = 5041

72^2 = 5184

73^2 = 5329

74^2 = 5476

75^2 = 5625

76^2 = 5776

77^2 = 5929

78^2 = 6084

79^2 = 6241

80^2 = 6400

81^2 = 6561

82^2 = 6724

83^2 = 6889

84^2 = 7056

85^2 = 7225

86^2 = 7396

87^2 = 7569

88^2 = 7744

89^2 = 7921

90^2 = 8100

91^2 = 8281

```
92^2 = 8464
93^2 = 8649
94^2 = 8836
95^2 = 9025
96^2 = 9216
97^2 = 9409
98^2 = 9604
99^2 = 9801
100^2 = 10000

squares_arr (первые 10): [1, 4, 9, 16, 25, 36, 49, 64, 81, 100] ...
```

5.3 Задание 2. Условный оператор: чётное/нечётное

Напишем функцию, которая печатает «чётное» для чётных чисел и «нечётное» — для нечётных. Реализуем двумя способами: через `if` и через тернарный оператор.

```
function task2()
    function check_parity_if(x)
        if x % 2 == 0
            println("$x — чётное")
        else
            println("$x — нечётное")
        end
    end

    check_parity_if(7)
    check_parity_if(10)
```

```
    check_parity_ternary(x) = println("$x — ", x % 2 == 0 ? "чётное" : "нечётное")
    check_parity_ternary(7)
    check_parity_ternary(10)
end

task2()
```

```
7 — нечётное
10 — чётное
7 — нечётное
10 — чётное
```

5.4 Задание 3. Функция add_one

Функция `add_one` добавляет 1 к своему аргументу.

```
function task3()
    add_one(x) = x + 1
    println("add_one(41) = ", add_one(41))
end

task3()
```

```
add_one(41) = 42
```

5.5 Задание 4. map / broadcast: увеличение элементов матрицы

Используем `broadcast` (или `map`) для увеличения каждого элемента матрицы на 1.

```
function task4()
    add_one(x) = x + 1
    A4x4 = reshape(1:16, 4, 4)
    println("Исходная матрица A4x4:")
    display(A4x4)
    println("\nadd_one.(A4x4) — каждый элемент +1:")
    display(add_one.(A4x4))
end

task4()
```

```
Исходная матрица A4x4:

add_one.(A4x4) — каждый элемент +1:

4×4 reshape(::UnitRange{Int64}, 4, 4) with eltype Int64:
 1  5   9  13
 2  6  10  14
 3  7  11  15
 4  8  12  16

4×4 Matrix{Int64}:
 2  6  10  14
 3  7  11  15
 4  8  12  16
 5  9  13  17
```

5.6 Задание 5. Матрица A, A^3, замена столбца

Дана матрица:

$$A = \begin{pmatrix} 1 & 1 & 3 \\ 5 & 2 & 6 \\ -2 & -1 & -3 \end{pmatrix}.$$

Требуется найти A^3 и заменить третий столбец на сумму второго и третьего.

```
function task5()
    A5x5 = [1 1 3; 5 2 6; -2 -1 -3]
    println("Матрица A:")
    display(A5x5)

    println("\nA^3:")
    display(A5x5^3)

    println("\nЗамена 3-го столбца на сумму 2-го и 3-го:")
    B5 = copy(A5x5)
    B5[:, 3] = A5x5[:, 2] + A5x5[:, 3]
    display(B5)
end

task5()
```

```
Матрица A:

A^3:

Замена 3-го столбца на сумму 2-го и 3-го:

3×3 Matrix{Int64}:
  1   1   3
```

```
  5   2   6
 -2  -1  -3

3×3 Matrix{Int64}:
 0  0  0
 0  0  0
 0  0  0

3×3 Matrix{Int64}:
  1   1   4
  5   2   8
 -2  -1  -4
```

5.7 Задание 6. Матрица B и C = B^T·B

Создадим матрицу B размера 15×2 с элементами $B_{i1} = 10$, $B_{i2} = -10$ для $i = 1, \ldots, 15$. Вычислим $C = B^T B$.

```
function task6()
    Bm = zeros(Int, 15, 2)
    for i in 1:15
        Bm[i, 1] = 10
        Bm[i, 2] = -10
    end
    println("Матрица B (первые 5 строк):")
    display(Bm[1:5, :])
    println("...")


    C = Bm' * Bm
    println("\nC = Bᵀ·B:")
```

```
    display(C)
end


task6()
```

```
Матрица B (первые 5 строк):
...

C = Bᵀ·B:

5×2 Matrix{Int64}:
 10  -10
 10  -10
 10  -10
 10  -10
 10  -10

2×2 Matrix{Int64}:
  1500  -1500
 -1500   1500
```

5.8 Задание 7. Матрицы Z_1..Z_4

Создадим матрицы 6×6 из нулей и единиц по заданным закономерностям. Вспомогательная функция `make_Z` принимает предикат и размер.

```
function task7()
    function make_Z(pattern::Function, n::Int)
        Z = zeros(Int, n, n)
        for i in 1:n
```

```
            for j in 1:n
                Z[i, j] = pattern(i, j) ? 1 : 0
            end
        end
        return Z
    end

    Z1 = make_Z((i, j) -> abs(i - j) == 1, 6)
    Z2 = make_Z((i, j) -> (i + j) % 2 == 0 && abs(i - j) <= 2, 6)
    Z3 = make_Z((i, j) -> (i + j) % 2 == 1 && 5 <= i + j <= 10, 6)
    Z4 = make_Z((i, j) -> (i + j) % 2 == 0, 6)

    println("Z1 (единицы на диагоналях):");  display(Z1)
    println("\nZ2 (шахматный, i+j чётное):");  display(Z2)
    println("\nZ3 (симметричная):");            display(Z3)
    println("\nZ4 (шахматный, i+j чётное):");  display(Z4)
end

task7()
```

```
Z1 (единицы на диагоналях):

Z2 (шахматный, i+j чётное):

Z3 (симметричная):

Z4 (шахматный, i+j чётное):

6×6 Matrix{Int64}:
```

```
 0  1  0  0  0  0
 1  0  1  0  0  0
 0  1  0  1  0  0
 0  0  1  0  1  0
 0  0  0  1  0  1
 0  0  0  0  1  0

6×6 Matrix{Int64}:
 1  0  1  0  0  0
 0  1  0  1  0  0
 1  0  1  0  1  0
 0  1  0  1  0  1
 0  0  1  0  1  0
 0  0  0  1  0  1

6×6 Matrix{Int64}:
 0  0  0  1  0  1
 0  0  1  0  1  0
 0  1  0  1  0  1
 1  0  1  0  1  0
 0  1  0  1  0  0
 1  0  1  0  0  0

6×6 Matrix{Int64}:
 1  0  1  0  1  0
 0  1  0  1  0  1
 1  0  1  0  1  0
 0  1  0  1  0  1
 1  0  1  0  1  0
 0  1  0  1  0  1
```

5.9 Задание 8. Функция outer() и матрицы $A_1..A_5$

Реализуем аналог функции `outer()` языка R: `outer(x, y, operation)` возвращает матрицу, где $C_{ij} = \text{operation}(x_i, y_j)$.

```
function task8()
    """
        outer(x, y, operation)


    Аналог функции outer() языка R.
    Возвращает матрицу C, где C[i, j] = operation(x[i], y[j]).
    """
    function outer(x, y, operation)
        m = length(x)
        n_ = length(y)
        C_ = Matrix{Any}(undef, m, n_)
        for i in 1:m
            for j in 1:n_
                C_[i, j] = operation(x[i], y[j])
            end
        end
        return C_
    end


    println("Проверка outer:")
    display(outer([1, 2, 3], [10, 20, 30], *))
    println()
    display(outer([1, 2, 3], [10, 20, 30], +))
```

```
    println("\nA1 = outer(0:4, 0:4, +):")
    display(outer(0:4, 0:4, +))

    println("\nA2 = outer(0:4, 0:4, i^j), 0^0=0:")
    display(outer(0:4, 0:4, (a, b) -> (a == 0 && b == 0) ? 0 : a^b))

    println("\nA3 = outer(0:4, 0:4, (i+j)%5):")
    display(outer(0:4, 0:4, (a, b) -> (a + b) % 5))

    println("\nA4 = outer(0:9, 0:9, (i+j)%10):")
    display(outer(0:9, 0:9, (a, b) -> (a + b) % 10))

    println("\nA5 = outer(0:8, 0:8, (j-i)%9):")
    display(outer(0:8, 0:8, (a, b) -> mod(b - a, 9)))
end

task8()
```

```
Проверка outer:


A1 = outer(0:4, 0:4, +):

A2 = outer(0:4, 0:4, i^j), 0^0=0:

A3 = outer(0:4, 0:4, (i+j)%5):

A4 = outer(0:9, 0:9, (i+j)%10):
```

```
A5 = outer(0:8, 0:8, (j-i)%9):

3×3 Matrix{Any}:
 10  20  30
 20  40  60
 30  60  90

3×3 Matrix{Any}:
 11  21  31
 12  22  32
 13  23  33

5×5 Matrix{Any}:
 0  1  2  3  4
 1  2  3  4  5
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8

5×5 Matrix{Any}:
 0  0   0   0    0
 1  1   1   1    1
 1  2   4   8   16
 1  3   9  27   81
 1  4  16  64  256

5×5 Matrix{Any}:
 0  1  2  3  4
 1  2  3  4  0
 2  3  4  0  1
 3  4  0  1  2
 4  0  1  2  3
```

```
10×10 Matrix{Any}:
 0  1  2  3  4  5  6  7  8  9
 1  2  3  4  5  6  7  8  9  0
 2  3  4  5  6  7  8  9  0  1
 3  4  5  6  7  8  9  0  1  2
 4  5  6  7  8  9  0  1  2  3
 5  6  7  8  9  0  1  2  3  4
 6  7  8  9  0  1  2  3  4  5
 7  8  9  0  1  2  3  4  5  6
 8  9  0  1  2  3  4  5  6  7
 9  0  1  2  3  4  5  6  7  8

9×9 Matrix{Any}:
 0  1  2  3  4  5  6  7  8
 8  0  1  2  3  4  5  6  7
 7  8  0  1  2  3  4  5  6
 6  7  8  0  1  2  3  4  5
 5  6  7  8  0  1  2  3  4
 4  5  6  7  8  0  1  2  3
 3  4  5  6  7  8  0  1  2
 2  3  4  5  6  7  8  0  1
 1  2  3  4  5  6  7  8  0
```

5.10 Задание 9. Решение СЛАУ Ax = y

Дана система линейных уравнений с 5 неизвестными:

$$\begin{cases} x_1 + 2x_2 + 3x_3 + 4x_4 + 5x_5 = 7, \\ 2x_1 + x_2 + 2x_3 + 3x_4 + 4x_5 = -1, \\ 3x_1 + 2x_2 + x_3 + 2x_4 + 3x_5 = -3, \\ 4x_1 + 3x_2 + 2x_3 + x_4 + 2x_5 = 5, \\ 5x_1 + 4x_2 + 3x_3 + 2x_4 + x_5 = 17. \end{cases}$$

Матрица A — симметричная тёплицева: $A_{ij} = |i - j| + 1$.

```
function task9()
    function build_A(n)
        A_ = zeros(Float64, n, n)
        for i in 1:n
            for j in 1:n
                A_[i, j] = abs(i - j) + 1
            end
        end
        return A_
    end

    y9 = [7.0, -1.0, -3.0, 5.0, 17.0]
    A9 = build_A(5)
    println("Матрица A (5x5):")
    display(A9)
    println("\nВектор y:")
    display(y9)

    x9 = A9 \ y9
    println("\nРешение x = A \\ y:")
```

```
    display(x9)
    println("\nПроверка A*x:")
    display(A9 * x9)

    A10 = build_A(10)
    y10 = ones(Float64, 10)
    x10 = A10 \ y10
    println("\nОтклонение A10*x10 - y10 для n=10: ", maximum(abs.(A10 * x10 - y10)))
end

task9()
```

```
Матрица A (5x5):

Вектор y:

Решение x = A \ y:

Проверка A*x:

Отклонение A10*x10 - y10 для n=10: 2.220446049250313e-16

5×5 Matrix{Float64}:
 1.0  2.0  3.0  4.0  5.0
 2.0  1.0  2.0  3.0  4.0
 3.0  2.0  1.0  2.0  3.0
 4.0  3.0  2.0  1.0  2.0
 5.0  4.0  3.0  2.0  1.0

5-element Vector{Float64}:
```

```
  7.0
 -1.0
 -3.0
  5.0
 17.0

5-element Vector{Float64}:
 -2.0000000000000013
  3.0000000000000027
  4.9999999999999964
  2.0000000000000027
 -4.000000000000001

5-element Vector{Float64}:
  6.999999999999999
 -1.0000000000000036
 -2.999999999999999
  4.999999999999998
 17.0
```

5.11 Задание 10. Матрица 6×10

Создадим матрицу M размера 6×10 из случайных целых чисел от 1 до 10 и выполним три подзадания.

```
function task10()
    Random.seed!(1234)
    M = rand(1:10, 6, 10)
    println("Матрица M:")
    display(M)
```

```
    N = 4
    counts = [count(>(N), M[i, :]) for i in 1:size(M, 1)]
    println("\nЧисло элементов > $N по строкам:")
    for i in 1:size(M, 1)
        println("Строка $i: ", counts[i])
    end

    Mtarget = 7
    rows_with_two = [i for i in 1:size(M, 1) if count(==(Mtarget), M[i, :]) == 2]
    println("\nСтроки, где $Mtarget встречается ровно 2 раза: ", rows_with_two)

    K = 75
    col_sums = [sum(M[:, j]) for j in 1:size(M, 2)]
    println("\nСуммы по столбцам: ", col_sums)

    pairs = Tuple{Int, Int}[]
    for j1 in 1:size(M, 2)
        for j2 in (j1+1):size(M, 2)
            if col_sums[j1] + col_sums[j2] > K
                push!(pairs, (j1, j2))
            end
        end
    end
    println("Пары столбцов с суммой > $K: ", pairs)
end

task10()
```

```
Матрица M:


Число элементов > 4 по строкам:
Строка 1: 8
Строка 2: 5
Строка 3: 6
Строка 4: 8
Строка 5: 8
Строка 6: 6


Строки, где 7 встречается ровно 2 раза: [4, 6]


Суммы по столбцам: [30, 45, 32, 34, 34, 35, 29, 33, 47, 40]
Пары столбцов с суммой > 75: [(1, 9), (2, 3), (2, 4), (2, 5), (2, 6), (2, 8), (2, 9), (2, 1

6×10 Matrix{Int64}:
 4  10   1  10  7   6  6   8   9   9
 6   8   2   8  3   4  1   3   8   5
 3   5   5   2  2   2  8   6   5  10
 9   8   7   5  7  10  1   3   9   6
 4   6  10   6  7   6  9   3   6   8
 4   8   7   3  8   7  4  10  10   2
```

5.12 Задание 11. Двойные суммы

Вычислим две двойные суммы:

$$S_1 = -\sum_{i=1}^{20}\sum_{j=1}^{5}\frac{i^4}{3+j},$$

$$S_2 = -\sum_{i=1}^{20}\sum_{j=1}^{5}\frac{i^4}{3+i\cdot j}.$$

```
function task11()
    S1 = -sum(i^4 / (3 + j) for i in 1:20, j in 1:5)
    println("S1 = ", S1)

    S2 = -sum(i^4 / (3 + i*j) for i in 1:20, j in 1:5)
    println("S2 = ", S2)
end

task11()
```

```
S1 = -639215.2833333338
S2 = -89912.02146097131
```

5.13 Выводы

В ходе работы были освоены и применены на практике:

- циклы `while` и `for` — для перебора элементов, работы со словарями и матрицами;
- условный оператор `if` и тернарный оператор — для проверки чётности;
- функции разных форм (обычные, однострочные, анонимные, с `!`);
- функции высшего порядка `map` и `broadcast` — для поэлементной обработки массивов;
- функция `outer()`, реализующая внешнее произведение с произвольной операцией;
- решение СЛАУ через оператор `\` с обобщением на разные размеры;
- генерация случайных матриц с воспроизводимостью через `Random.seed!`;

- двойные суммы через генераторы.

Все задания 1–11 выполнены, результаты совпадают с ожидаемыми.

6 Обсуждение результатов

- Все 11 заданий выполнены, вывод совпадает с ожидаемым.
- Матрицы Z_1–Z_4 и A_1–A_5, построенные через `outer()`, совпадают с образцами из методических указаний.
- СЛАУ с 5 неизвестными решена: $x = (-2, 3, 5, 2, -4)^T$; проверка $Ax \approx y$ даёт отклонение $\sim 10^{-15}$.
- Обобщение метода на $n = 10$: отклонение $Ax - y$ составляет $2.2 \cdot 10^{-16}$.
- Двойные суммы: $S_1 = -639215{,}28$, $S_2 = -89912{,}02$.

Результаты соответствуют теоретическим ожиданиям. Пакет `Literate` позволил из одного файла получить отчёт в двух форматах.

7 Заключение

Цель работы достигнута. Освоены циклы, условный и тернарный операторы, функции разных форм, `map`/`broadcast`, реализована функция `outer()`, решена СЛАУ с симметричной тёплицевой матрицей, применены пакеты `DrWatson` и `Literate`.

Список литературы

1. *Datseris G., contributors.* Literate.jl: Literate programming for Julia. — 2024. — URL: https://github.com/fredrikekre/Literate.jl.
2. DrWatson: A scientific project assistant for Julia / G. Datseris [et al.]. — 2023. — URL: https://github.com/JuliaDynamics/DrWatson.jl.
3. *Dunning I., Schubert L., Thorne J.* Programming Julia: High-Performance Applications. — Packt Publishing, 2019. — ISBN 978-1788296717.
4. Julia: A Fresh Approach to Numerical Computing. Vol. 59 / J. Bezanson [et al.]. — 2017. — P. 65–98. — DOI: 10.1137/141000671.
5. *Kwong T.* Hands-on Design Patterns and Best Practices with Julia. — Packt Publishing, 2020. — ISBN 978-1838646615.
6. *Newham C.* Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 p. — (In a Nutshell). — ISBN 0596009658. — URL: http://www.amazon.com/Learning-bash-Shell-Programming-Nutshell/dp/0596009658.
7. *Robbins A.* Bash Pocket Reference. — O'Reilly Media, 2016. — 156 p. — ISBN 978-1491941591.
8. *Zarrelli G.* Mastering Bash. — Packt Publishing, 2017. — 502 p. — ISBN 9781784396879.
9. *Таненбаум Э., Бос Х.* Современные операционные системы.. — 4-е изд. — СПб. : Питер, 2015.. — 1120 с. — (Классика Computer Science).